

UNI-TEL Five-Day Software Engineering Project Master Plan

Five-member execution plan for 14-18 August 2026

UNI-TEL Project Team

Version 1.0 - 14 August 2026

Contents

1	Document Control	4
2	How to Use This Plan	5
3	Project Integrity and Baseline Policy	6
3.1	Required baseline action	6
3.2	Required baseline note	6
3.3	Evidence rules	6
4	Project Definition	7
4.1	Problem statement	7
4.2	Product vision	7
4.3	Objectives	7
4.4	Stakeholders	7
4.5	In-scope product capabilities	8
4.6	Explicitly out of scope	8
4.7	Constraints and assumptions	8
5	Success Criteria	9
6	Methodology Coverage and Selection	10
6.1	Methodologies to explain in the report	10
6.2	Selected lifecycle statement	10
6.3	Five-day lifecycle map	11
7	Team Organization	12
7.1	Primary roles	12
7.2	Fair-contribution rule	12
7.3	RACI matrix	12
7.4	Review rotation	13
8	Project Governance	14
8.1	Daily cadence in IST	14
8.2	Decision authority	14
8.3	Definition of Ready	14
8.4	Definition of Done	14
9	Repository and Configuration Management	16
9.1	Target repository structure	16
9.2	Branch naming	16
9.3	Commit conventions	17
9.4	Pull-request requirements	17
9.5	Kanban board	17
9.6	Version and change control	17
10	Work Breakdown Structure	18
11	Prioritized Product and SE Backlog	19
12	Five-Day Execution Schedule	21
12.1	Day 1 - Friday, 14 August 2026	21

12.2 Day 2 - Saturday, 15 August 2026	22
12.3 Day 3 - Sunday, 16 August 2026	23
12.4 Day 4 - Monday, 17 August 2026	24
12.5 Day 5 - Tuesday, 18 August 2026	25
13 Required Software Engineering Artifact Set	26
14 Requirements Engineering Standard	28
14.1 ID scheme	28
14.2 Seed functional requirements	28
14.3 Seed non-functional requirements	29
14.4 Requirement quality checklist	29
14.5 Critical use cases	29
15 Analysis and Design Standard	31
15.1 Architecture boundary	31
15.2 Required diagrams and content	31
15.3 Design consistency rules	31
15.4 Database review focus	32
16 Implementation and Test Strategy	33
16.1 Product-hardening principle	33
16.2 Planned automated test areas	33
16.3 Test levels and types	33
16.4 Test-case record	34
16.5 Defect severity	34
16.6 Quality commands	34
17 Traceability Strategy	35
18 Project Estimation and Resource Plan	36
19 Risk Management	37
20 Change Management	38
21 Quality Assurance Plan	39
21.1 Review checklists	39
21.2 Project metrics	39
22 Deployment, Operations, and Maintenance	40
22.1 Environment model	40
22.2 Deployment guide must include	40
22.3 Maintenance categories	40
23 Final Report Structure	41
24 Presentation and Demonstration Plan	42
24.1 Recommended 18-slide deck	42
24.2 Theory-to-evidence rule	42
24.3 Timed demo script	42
24.4 Viva coverage by every member	42
25 Evidence and Contribution Management	44
25.1 Daily evidence folders	44
25.2 Contribution matrix	44
26 Final Submission Structure	45
27 Final Audit Checklists	46
27.1 SE completeness	46
27.2 Repository and release	46
27.3 Presentation readiness	46

28 Reusable Record Templates	47
28.1 Daily progress record	47
28.2 Meeting minutes record	47
28.3 Architecture decision record	47
28.4 Change request example	47
28.5 Defect example	48

29 Immediate Start Checklist	49
-------------------------------------	-----------

30 Approval	50
--------------------	-----------

For the project team: Execute this plan through GitHub issues and pull requests. Track every checkbox, attach genuine evidence, and do not mark an activity complete until its stated exit criteria are satisfied.

Goal: Convert the imported UNI-TEL prototype into a traceable, tested, documented, and presentable Software Engineering class project in five calendar days with five team members.

Approach: Use Rapid Application Development (RAD) as the primary lifecycle, iterative and incremental delivery for daily integration, Kanban for task flow, prototyping for UI validation, V-Model-style requirement-to-test traceability, and DevOps practices for automated quality checks and release management.

Technology baseline: React 18, TypeScript, Vite, Supabase Authentication and PostgreSQL, Tailwind CSS, shadcn/ui, TanStack Query, Recharts, Git, GitHub, GitHub Actions, Vitest, React Testing Library, Vercel, Markdown, LaTeX.

Official execution window: Friday, 14 August 2026 through Tuesday, 18 August 2026 (IST).

Chapter 1

Document Control

Field	Value
Project	UNI-TEL - Student Academic Management and Analytics Platform
Document	Five-Day Software Engineering Master Plan
Version	1.0
Planning date	14 August 2026
Duration	5 calendar days
Team size	5 members
Primary methodology	Rapid Application Development
Supporting practices	Iterative delivery, Kanban, prototyping, V-Model traceability, DevOps
Repository baseline	Existing UNI-TEL code imported as one clearly identified baseline commit
Final release target	v1.0.0-se-class

Replace the labels Member 1 through Member 5 with actual names in the Project Charter on Day 1. Keep the role labels in addition to names so ownership remains clear.

Chapter 2

How to Use This Plan

1. Create the new GitHub repository and import the existing application as one baseline commit.
2. Create one GitHub milestone named **UNI-TEL SE Class - 18 Aug 2026**.
3. Create issues from the work-package and backlog tables in this plan.
4. Assign one accountable owner and one reviewer to each issue.
5. Move issues across the Kanban board as evidence is produced.
6. Use feature, documentation, test, fix, or CI branches. Do not work directly on **main**.
7. Submit each coherent change through a pull request linked to its issue and requirement IDs.
8. Run the daily review gate before ending each day.
9. Update the requirements traceability matrix continuously, not only on Day 5.
10. Complete the final release gate before tagging the submission.

Chapter 3

Project Integrity and Baseline Policy

The new repository represents the official five-day class effort. The application code already exists, so its origin must be handled transparently.

3.1 Required baseline action

The first repository commit must be:

```
chore: import UNI-TEL baseline prototype
```

The baseline commit may contain the current application code, but it must exclude the former `.git` directory, `node_modules`, `.env`, credentials, generated build output, local editor state, and temporary files.

3.2 Required baseline note

Add this statement to the new repository README:

This repository began with an imported UNI-TEL application prototype. During the official five-day Software Engineering project window, the team formalized requirements and design, established traceability, added testing and quality controls, corrected verified defects, documented deployment and maintenance, and prepared the assessed release.

3.3 Evidence rules

- Do not backdate commits, forge interviews, invent user feedback, or manufacture test results.
- Do not split old source files into artificial commits to simulate implementation history.
- Record only work genuinely completed during the five-day period.
- Mark a test Pass only after execution and preservation of its result.
- Mark a review complete only after another member provides a substantive review.
- Preserve citations and licenses for reused libraries, components, templates, and assets.
- Distinguish measured values from targets. For example, a performance goal is not a measured result until a test report exists.

Chapter 4

Project Definition

4.1 Problem statement

Students often manage semesters, subjects, marks, attendance, and academic progress using disconnected notebooks, spreadsheets, portals, or calculators. Fragmented data makes it difficult to calculate SGPA and CGPA consistently, identify attendance shortages, evaluate trends, and generate useful reports. UNI-TEL provides one secure, responsive platform in which an individual student can maintain and analyze academic records.

4.2 Product vision

UNI-TEL will be a student-centered academic tracking application that converts raw marks and attendance data into understandable progress information while preserving user ownership, data integrity, and portability.

4.3 Objectives

- Centralize semester, subject, marks, and attendance information.
- Calculate attendance percentage, SGPA, CGPA, and weighted marks consistently.
- Help students recognize weak attendance and performance trends.
- Provide analytics and exportable academic reports.
- Isolate each user's data through authentication and database security.
- Demonstrate a complete and traceable Software Engineering lifecycle.
- Deliver a reproducible build, tested release, user guide, and maintenance plan.

4.4 Stakeholders

Stakeholder	Interest	Influence	Engagement
Student user	Accurate, private, usable academic tracking	High	Primary requirements and acceptance viewpoint
Course professor	Evidence of SE theory and disciplined execution	High	Reviews final artifacts and demonstration
Five-member project team	Deliverable quality and fair contribution	High	Daily planning, implementation, review, presentation
Supabase	Backend platform and service constraints	Medium	Architecture and operational dependency
Vercel or selected host	Frontend deployment	Medium	Deployment and release dependency
Future teachers/institutions	Possible multi-user extension	Low	Future scope only

4.5 In-scope product capabilities

- Registration, login, logout, session handling, and protected routes.
- Student profile management.
- Semester creation, update, viewing, and deletion.
- Subject and credit management.
- Attendance recording and percentage calculation.
- Examination marks, total marks, weightage, and percentage management.
- SGPA and CGPA calculation.
- Performance dashboards, trends, and academic analytics.
- JSON import/export and PDF/Excel reporting where the baseline supports them.
- User notifications where the baseline supports them.
- Responsive web interface, validation, error handling, and empty/loading states.
- Supabase schema, constraints, migrations, authentication, and Row-Level Security.
- Automated quality checks, test evidence, deployment documentation, and release packaging.

4.6 Explicitly out of scope

- Teacher, administrator, and institution-wide dashboards.
- Knowledge Hub or study-material sharing.
- Social networking, groups, leaderboards, or chat.
- AI recommendations or grade prediction presented as production AI.
- Native Android or iOS applications.
- Payment processing, LMS integration, and bulk institutional imports.
- A complete redesign of the existing application.
- Unverified claims of production scale, guaranteed uptime, WCAG certification, or benchmark performance.

4.7 Constraints and assumptions

- The project has five calendar days and five contributors.
- The imported prototype is the technical baseline.
- Internet access is required for hosted Supabase and deployment services.
- The team will use test or synthetic academic data only.
- The final system is an academic prototype, not a certified production service.
- Scope changes require a recorded change request and Project Lead approval.
- `supabase/migrations` is the authoritative schema source until the team verifies otherwise.

Chapter 5

Success Criteria

The project is successful only when all of the following are true:

1. The scope, objectives, stakeholders, feasibility, and methodology are documented.
2. Every approved functional and non-functional requirement has a stable identifier.
3. Major requirements have use cases and measurable acceptance criteria.
4. Architecture, behavior, database, component, and deployment views are documented consistently.
5. Each Must-have requirement maps to implementation and at least one executed test in the RTM.
6. Critical calculations and validation rules have automated tests.
7. Authentication, authorization, data integrity, error handling, and responsive behavior have test evidence.
8. CI runs installation, type checking, linting, automated tests, and production build.
9. All Critical and High release-blocking defects are closed or formally accepted with justification.
10. The application builds reproducibly and the deployment procedure is documented.
11. PRD, SRS, design, test, deployment, maintenance, user, and final-report artifacts agree with one another.
12. Every team member has attributable issues, commits, pull requests, reviews, technical work, documentation work, and a presentation segment.
13. The final release is tagged **v1.0.0-se-class** and accompanied by release notes.
14. The presentation demonstrates theory through direct project evidence, not definitions alone.

Chapter 6

Methodology Coverage and Selection

6.1 Methodologies to explain in the report

Method or model	Core idea	Strength	Limitation	UNI-TEL decision
Waterfall	Complete sequential phases	Clear documents and phase gates	Change is expensive and feedback arrives late	Explain for theory; do not claim as primary
V-Model	Pair development stages with test levels	Strong verification and validation traceability	Rigid when requirements change	Use requirement-to-test mapping only
Prototyping	Learn through early UI or behavior models	Fast feedback and requirement clarification	Prototype may be mistaken for production quality	Use the imported UI as the starting prototype
Incremental	Deliver functionality in usable slices	Early value and easier integration	Requires disciplined interfaces	Use daily integrated increments
Iterative	Revisit and improve earlier work	Supports discovery and correction	Can cause uncontrolled rework	Use daily review and refinement
Spiral	Risk-driven cycles	Strong for large, uncertain, high-risk systems	Too costly and complex for five days	Explain and reject for this scope
RAD	Time-boxed construction with reuse and prototyping	Fits short deadlines and existing components	Documentation and architecture can be neglected	Primary lifecycle with mandatory documentation gates
Agile	Adaptive delivery and stakeholder feedback	Visibility and responsiveness	Can be misused as no-documentation development	Use values, backlog, reviews, and iteration
Scrum	Roles, events, and fixed sprints	Predictable team cadence	Five days is too short for a meaningful Scrum program	Do not claim five one-day sprints
Kanban	Visualize flow and limit work in progress	Ideal for short continuous flow	Less prescriptive about planning	Use for daily work management
DevOps	Integrate development, testing, release, and operations	Repeatable quality and deployment	Requires automation discipline	Use CI, environment control, release, and monitoring plan

6.2 Selected lifecycle statement

Use the following wording consistently in the report and presentation:

UNI-TEL used Rapid Application Development as the primary five-day lifecycle. The team organized

work as daily iterative and incremental deliveries, managed flow through Kanban, used the imported interface as a prototype for validation, linked requirements to tests using V-Model-style traceability, and applied DevOps practices for continuous integration and controlled release.

6.3 Five-day lifecycle map

Day 1: Initiation, feasibility, baseline audit, scope, backlog

- > Day 2: Requirements engineering and high-level analysis
- > Day 3: Detailed design, test-first improvements, CI integration
- > Day 4: System integration, verification, deployment rehearsal
- > Day 5: Regression, release, evaluation, presentation
- > Maintenance: controlled future changes, monitoring, support

This sequence displays all classical SDLC phases while remaining honest about the compressed, iterative execution.

Chapter 7

Team Organization

7.1 Primary roles

Member	Primary role	Secondary technical ownership	Main documentation ownership	Presentation ownership
Member 1	Project Lead and Product Owner	Authentication/security audit and approved fixes	Charter, scope, PRD, feasibility, backlog	Problem, planning, feasibility, methodology
Member 2	Business Analyst and UX Lead	Form validation, accessibility, responsive/UI fixes	SRS, use cases, UI specification	Requirements, use cases, UI
Member 3	Solution Architect and Data Lead	Supabase schema, migrations, integrity, database fixes	SDD, architecture, UML/DFD, ERD, data dictionary	Architecture, database, design
Member 4	QA and Security Test Lead	Test framework, automated tests, security/system tests	Test plan, cases, defect log, RTM	Testing, traceability, quality
Member 5	Configuration, DevOps, and Release Lead	CI, environments, build, deployment, release	SCM plan, deployment, maintenance, release notes	Git, CI/CD, deployment, maintenance

7.2 Fair-contribution rule

Every member must complete all four categories:

- At least one technical contribution or verified technical analysis.
- At least one authored or co-authored SE artifact.
- At least one pull request reviewed for another member.
- One final presentation segment and participation in the demo or questions.

Do not use commit count as the only measure of contribution. Use completed issues, complexity, authored artifacts, tests, reviews, presentation work, and evidence quality.

7.3 RACI matrix

Key: A = Accountable, R = Responsible, C = Consulted, I = Informed.

Activity	M1	M2	M3	M4	M5
Scope and charter	A/R	C	C	C	C
PRD and backlog	A/R	R	C	C	I
SRS and use cases	A	A/R	C	C	I

Activity	M1	M2	M3	M4	M5
Architecture and database design	C	C	A/R	C	C
UI/UX specification	C	A/R	C	C	I
Test strategy and RTM	C	C	C	A/R	C
Security assessment	R	C	R	A	C
Configuration and CI	I	I	C	C	A/R
Deployment and release	A	I	C	C	A/R
Final report integration	A/R	R	R	R	R
Presentation and demonstration	A	R	R	R	R

7.4 Review rotation

Author	Primary reviewer	Backup reviewer
Member 1	Member 2	Member 4
Member 2	Member 3	Member 1
Member 3	Member 4	Member 5
Member 4	Member 5	Member 3
Member 5	Member 1	Member 2

The primary reviewer checks correctness and traceability. The backup reviewer is used for high-risk schema, authentication, CI, or release changes.

Chapter 8

Project Governance

8.1 Daily cadence in IST

Time	Event	Maximum duration	Required output
09:00	Daily stand-up	15 min	Yesterday/today/blockers note
09:15	Planning and task confirmation	15 min	Updated owners and board
13:00	Integration checkpoint	20 min	Early conflicts and dependency update
17:30	Pull-request and artifact review	45 min	Review comments and approvals
18:15	Daily system integration	30 min	Green main branch or rollback decision
18:45	Daily review and retrospective	30 min	Gate result, risk update, next-day priorities

If the team uses different working hours, preserve the event order and outputs.

8.2 Decision authority

- Product scope and priority: Member 1 after team consultation.
- Requirement wording: Member 2, approved by Member 1.
- Architecture and schema: Member 3, reviewed by Members 4 and 5.
- Release quality and test status: Member 4; Member 1 accepts residual academic-project risk.
- CI, deployment, and versioning: Member 5, approved by Member 1.
- Any scope increase after Day 2: formal change request required.

8.3 Definition of Ready

An issue may enter **In Progress** only when it has:

- A clear title and one accountable owner.
- A relevant work-package or requirement ID.
- Testable acceptance criteria.
- Known dependencies and reviewer.
- A size small enough to complete within one day.
- Evidence expectations, such as a document section, test output, diagram, screenshot, or deployment URL.

8.4 Definition of Done

An issue is **Done** only when:

- Acceptance criteria are satisfied.
- Requirement and design references are updated where relevant.

- Tests have been written and executed at the appropriate level.
- Type check, lint, automated tests, and production build pass.
- No secrets, personal data, generated junk, or unrelated changes are committed.
- A teammate has reviewed the pull request.
- User-facing or operational documentation is updated.
- Evidence is linked to the issue.
- The change is merged and verified on `main`.

Chapter 9

Repository and Configuration Management

9.1 Target repository structure

```
UNI-TEL/  
|-- .github/  
|   |-- ISSUE_TEMPLATE/  
|   |-- workflows/quality.yml  
|   `-- pull_request_template.md  
|-- docs/  
|   |-- 01-initiation/  
|   |-- 02-requirements/  
|   |-- 03-design/  
|   |-- 04-testing/  
|   |-- 05-project-management/  
|   |-- 06-deployment-maintenance/  
|   |-- 07-user-guide/  
|   |-- 08-final-report/  
|   |-- 09-presentation/  
|   |-- evidence/day-1/ ... day-5/  
|   `-- planning/  
|-- src/  
|-- supabase/  
|-- tests/  
|   |-- unit/  
|   |-- integration/  
|   `-- fixtures/  
|-- output/pdf/  
|-- .env.example  
|-- .gitignore  
|-- CONTRIBUTING.md  
|-- LICENSE  
|-- README.md  
`-- package.json
```

9.2 Branch naming

```
docs/WP1-charter-prd  
docs/WP2-srs-use-cases  
design/WP3-architecture-database  
test/WP5-grade-validation  
fix/WP4-schema-consistency
```

```
feat/WP4-accessibility-validation
ci/WP6-quality-pipeline
release/v1.0.0-se-class
```

9.3 Commit conventions

Use Conventional Commit style and one coherent purpose per commit:

```
docs(charter): define scope stakeholders and constraints
docs(srs): specify FR-01 through FR-12
design(database): add verified ERD and data dictionary
test(grades): cover SGPA and weighted marks boundaries
fix(validation): reject obtained marks above total marks
fix(schema): align schema documentation with migrations
feat(ally): label marks and attendance form controls
ci: verify types lint tests and production build
docs(rtm): map approved requirements to tests and evidence
release: prepare v1.0.0 SE class submission
```

Aim for coherent commits, not a quota. Normal merge commits should preserve individual authors when contribution history is assessed.

9.4 Pull-request requirements

Every PR must contain:

- Linked issue using Closes `#number` when appropriate.
- Work-package and requirement IDs.
- Concise summary and explicit out-of-scope statement.
- Verification commands and actual results.
- Screenshots for visible UI changes.
- Migration and rollback notes for database changes.
- Documentation and RTM impact.
- At least one approving reviewer.

9.5 Kanban board

Use these columns and WIP limits:

Column	Meaning	WIP limit
Product Backlog	Approved but not scheduled	None
Selected	Ready for current day	10
In Progress	Actively owned	1 per member
In Review	PR or document awaiting review	5
Testing	Integrated verification	5
Done	Meets Definition of Done	None
Blocked	Waiting for dependency or decision	None

9.6 Version and change control

- Baseline: `0.1.0-baseline` after imported prototype verification.
- Candidate: `1.0.0-rc.1` after Day 4 integration.
- Final: `v1.0.0-se-class` after Day 5 release gate.
- Documents carry version, author, reviewer, approval date, and change history.
- Schema changes use new forward migrations; do not silently edit an already-applied migration.
- Secrets exist only in local or hosted environment settings, never in Git.

Chapter 10

Work Breakdown Structure

WP	Work package	Main outcome	Owner	Primary reviewer	Due
WP0	Repository and baseline governance	Clean baseline, board, templates, contribution rules	M5	M1	Day 1
WP1	Initiation and product planning	Charter, feasibility, PRD, scope, backlog	M1	M2	Day 1
WP2	Requirements engineering	SRS, actors, use cases, NFRs, acceptance criteria	M2	M3	Day 2
WP3	Analysis and design	SDD, UML, DFD, ERD, data dictionary, UI/security design	M3	M4	Day 3
WP4	Targeted product hardening	Verified fixes for security, validation, schema, UX	M1/M2/ M3	M4	Day 4
WP5	Verification and validation	Automated/manual tests, defect log, RTM, report	M4	M5	Day 4
WP6	CI/CD and release engineering	Quality workflow, environments, deploy, release/rollback	M5	M1	Day 5
WP7	Management controls	Gantt, estimates, risks, changes, meetings, metrics	M1	M5	Daily
WP8	Final report and presentation	Integrated report, slides, demo, viva, release package	All	M1	Day 5

Chapter 11

Prioritized Product and SE Backlog

Use MoSCoW priority. Story points are relative planning values, not elapsed hours.

ID	Backlog item	Priority	Points	Owner	Reviewer	Target	Dependency
SE-001	Import and verify clean baseline	Must	3	M5	M1	D1	None
SE-002	Create charter, scope, objectives, stakeholder register	Must	5	M1	M2	D1	SE-001
SE-003	Complete five-part feasibility study	Must	3	M1	M3	D1	SE-002
SE-004	Compare lifecycle models and record RAD decision	Must	3	M1	M4	D1	SE-002
SE-005	Audit features, code, schema, tests, docs, security, deployment	Must	8	All	M1	D1	SE-001
SE-006	Configure board, templates, contribution and review policy	Must	5	M5	M1	D1	SE-001
SE-007	Write PRD and prioritized product backlog	Must	8	M1	M2	D2	SE-005
SE-008	Write SRS with FR/NFR identifiers and business rules	Must	13	M2	M3	D2	SE-007
SE-009	Define actors, use cases, acceptance criteria, and UI flows	Must	8	M2	M1	D2	SE-008
SE-010	Create context, use-case, DFD 0/1, activity diagrams	Must	8	M3	M2	D3	SE-008
SE-011	Create architecture, component, sequence, deployment diagrams	Must	8	M3	M4	D3	SE-008
SE-012	Verify ERD, schema, constraints, RLS, and data dictionary	Must	13	M3	M4	D3	SE-005
SE-013	Create test strategy, test data, cases, and RTM skeleton	Must	8	M4	M5	D2	SE-008
SE-014	Add automated test framework and calculation/validation tests	Must	13	M4	M5	D3	SE-013
SE-015	Add type-check/test scripts and GitHub Actions quality gate	Must	8	M5	M4	D3	SE-014
SE-016	Correct verified security/authentication issues	Must if found	8	M1	M4	D4	SE-005
SE-017	Correct verified validation/accessibility/responsive issues	Must if found	8	M2	M4	D4	SE-005

ID	Backlog item	Priority	Points	Owner	Reviewer	Target	Dependency
SE-018	Correct verified schema/integrity/documentation issues	Must if found	8	M3	M4	D4	SE-012
SE-019	Execute unit, integration, system, security, UX tests	Must	13	M4	M1	D4	SE-014 to 018
SE-020	Manage defect triage, retest, regression, and RTM completion	Must	8	M4	M5	D5	SE-019
SE-021	Document configuration, deployment, rollback, maintenance	Must	8	M5	M1	D4	SE-015
SE-022	Deploy release candidate and preserve evidence	Must	5	M5	M1	D4	SE-015, SE-021
SE-023	Create user manual and installation guide	Must	5	M2/M5	M3	D5	SE-022
SE-024	Integrate final report and appendices	Must	13	All	M1	D5	SE-002 to 023
SE-025	Prepare slides, demo, viva, and contribution report	Must	8	All	M1	D5	SE-024
SE-026	Tag release and archive final submission	Must	3	M5	M1	D5	SE-020, SE-025

The team should not begin Could-have features until every Must item required for the release gate is complete.

Chapter 12

Five-Day Execution Schedule

12.1 Day 1 - Friday, 14 August 2026

Theme: Initiation, transparent baseline, feasibility, scope, and governance.

Team objectives: Establish the official repository, understand the imported system, decide what will and will not be delivered, assign ownership, and create an auditable plan.

Member	Assigned work	Required evidence
M1	Draft charter, problem, vision, objectives, stakeholders, scope, assumptions, constraints; lead methodology decision	Charter PR, decision record ADR-001, issue links
M2	Audit current user flows, forms, pages, responsive behavior, accessibility; identify actors and preliminary use cases	UI audit with screenshots and issue list
M3	Audit architecture, services, migrations, entities, constraints, RLS; flag inconsistencies	Architecture/database audit and verified schema notes
M4	Audit existing tests, calculation risks, validation, security scenarios, and quality claims	Test-gap report and initial risk/test inventory
M5	Create clean baseline, <code>.gitignore</code> , <code>.env.example</code> , templates, board, milestone, branch policy, build audit	Baseline tag, repository settings evidence, build log

Whole-team activities:

- Demonstrate the baseline from login through analytics/export.
- Agree on scope freeze and selected methodology.
- Convert audit findings into prioritized issues.
- Assign one owner and reviewer to every Day 2 issue.
- Establish document naming, ID conventions, and evidence locations.

Required outputs:

- Project Charter v1.0.
- Stakeholder Register.
- Technical, economic, operational, schedule, and legal/security feasibility study.
- Methodology comparison and ADR-001 selecting RAD.
- Scope statement and feature freeze.
- Initial product/SE backlog and five-day Gantt chart.
- Risk Register v0.1.
- Baseline audit reports.
- Repository contribution, review, and configuration rules.

Suggested commits:

```

chore: import UNI-TEL baseline prototype
chore(repo): add contribution and review controls
docs(charter): define project scope and stakeholders
docs(feasibility): assess five-day prototype viability
docs(methodology): select RAD with traceability practices
docs(audit): record baseline UI architecture test and release gaps

```

Day 1 exit gate: Baseline builds; no secrets are tracked; scope is frozen; methodology is approved; responsibilities and reviewers are assigned; Day 2 issues satisfy Definition of Ready.

12.2 Day 2 - Saturday, 15 August 2026

Theme: Requirements engineering, analysis, acceptance, and test planning.

Team objectives: Convert the product vision and baseline behavior into approved, numbered, measurable requirements that can drive design and testing.

Member	Assigned work	Required evidence
M1	Complete PRD, personas/user goals, business value, MoSCoW priorities, release criteria	PRD PR and backlog mapping
M2	Complete SRS, actors, FRs, NFRs, business rules, detailed use cases, acceptance criteria	SRS PR with review comments resolved
M3	Create system context and initial logical data/process models; verify requirement feasibility against architecture	Context/DFD drafts and feasibility comments
M4	Create test strategy, test levels/types, entry/exit criteria, test environment, test data, RTM skeleton	Test Plan v0.1 and initial RTM
M5	Create SCM plan, CI design, environment inventory, deployment architecture draft, backup/rollback plan	SCM/DevOps documents and workflow issue

Formal requirements review: At 17:30, read each Must requirement. Check that it is necessary, singular, unambiguous, feasible, testable, uniquely identified, prioritized, and traceable.

Required outputs:

- PRD v1.0.
- SRS v1.0.
- Actor and use-case catalogue.
- Use-case diagram and detailed descriptions for critical workflows.
- Functional and non-functional requirement catalogues.
- Business rules and data-validation rules.
- MoSCoW priority matrix.
- Acceptance criteria.
- Test Plan v0.1 and RTM v0.1.
- Requirements Review Minutes and approved baseline.

Suggested commits:

```

docs(prd): define users value scope and release outcomes
docs(srs): specify functional and nonfunctional requirements
docs(use-cases): model core student workflows
test(plan): define verification strategy and RTM structure
docs(scm): define branches environments builds and releases

```

Day 2 exit gate: All Must requirements are approved and testable; requirement IDs are frozen; every Must requirement has acceptance criteria and planned tests; open ambiguity is recorded as a decision or change request.

12.3 Day 3 - Sunday, 16 August 2026

Theme: Detailed design, test-first product hardening, automation, and design review.

Team objectives: Produce consistent architecture and data designs, add the missing automated quality foundation, and implement only verified high-value corrections.

Member	Assigned work	Required evidence
M1	Verify authentication/session/authorization design; write failing security/route scenarios with M4; implement approved fix if a failure exists	Issue, test evidence, reviewed PR
M2	Finalize UI specification and activity flows; write validation/accessibility scenarios; implement approved issue using test-first steps	Before/after evidence and reviewed PR
M3	Complete SDD, ERD, data dictionary, constraints/RLS, UML/DFD; fix only verified schema or schema-documentation defects	Design package and migration/verification evidence
M4	Configure Vitest/RTL; add unit tests for grade, attendance, marks, semester normalization, and validation services	Failing-then-passing test commits and coverage output
M5	Add type-check/test scripts and GitHub Actions workflow; verify clean install and production build	Green workflow link/log and failure-handling notes

Required design views: context, use-case, DFD Level 0, DFD Level 1, activity, sequence, component, ER, deployment, and high-level architecture. Every diagram must contain a title, legend where needed, consistent names, and a paragraph explaining how it relates to requirements.

Test-first change cycle:

1. Link an issue to a requirement or verified audit finding.
2. Add a test or reproducible failure procedure.
3. Demonstrate that the check fails for the intended reason.
4. Apply the smallest correction.
5. Demonstrate that the check passes and no regression appears.
6. Update documentation and RTM.
7. Obtain peer review before merge.

Required outputs:

- Software Design Document v1.0.
- Complete diagram set and data dictionary.
- UI, validation, error-handling, and security design sections.
- Automated test framework and initial critical tests.
- CI quality workflow.
- Verified technical correction PRs, if failures were found.
- Architecture and Design Review Minutes.

Suggested commits:

```

design(architecture): document component and deployment views
design(database): add verified ERD constraints RLS and dictionary
test(setup): configure Vitest and React Testing Library
test(calculations): cover SGPA attendance and marks boundaries
test(import): cover semester normalization regressions
ci: run type check lint tests and build on pull requests
fix(...): correct the verified behavior described by the linked issue

```

Day 3 exit gate: Design names agree with SRS and code; critical automated tests pass; CI is green; every correction has failure evidence and review; design changes are reflected in the RTM.

12.4 Day 4 - Monday, 17 August 2026

Theme: Integration, system verification, defect resolution, and release rehearsal.

Team objectives: Verify complete user journeys and non-functional targets, close release-blocking defects, complete deployment documentation, and create the release candidate.

Member	Assigned work	Required evidence
M1	Execute acceptance tests for scope and critical journeys; triage defects; control scope/change requests	UAT record, decisions, updated risk register
M2	Execute usability, accessibility, responsive, browser, form-validation, and user-manual walkthrough tests	Device/browser matrix, screenshots, issue results
M3	Execute database CRUD, constraint, cascade, calculation, migration, and RLS integration tests	Query/test log using synthetic users and data
M4	Coordinate unit, integration, system, regression, security, negative, recovery, and exploratory testing; maintain defect log and RTM	Test execution report, defect dashboard, RTM v0.9
M5	Run clean build, environment and deployment rehearsal; deploy 1.0.0-rc.1; test rollback and document operations	Deployment URL/evidence, build log, rollback record

Core end-to-end scenarios:

1. Register/login and reach protected dashboard.
2. Create semester and add subjects/credits.
3. Enter valid and invalid attendance data.
4. Enter valid and invalid marks/weightage.
5. Verify calculated percentages, SGPA, and CGPA.
6. View analytics and empty/error/loading states.
7. Import valid data and reject malformed or unsafe data.
8. Export supported report formats.
9. Verify logout and protected-route behavior.
10. Verify User A cannot access User B's records.

Required outputs:

- Executed test-case register with actual results.
- Automated test and CI reports.
- Defect Log with severity, priority, owner, resolution, retest.
- Security, usability, compatibility, and data-integrity evidence.
- RTM v0.9.
- Deployment Guide, rollback plan, maintenance plan, and operations checklist.
- Release Candidate 1.0.0-rc.1 and rehearsal record.

Suggested commits:

```
test(system): record core workflow verification
test(security): verify protected routes and user isolation
test(database): verify constraints cascades and RLS
fix(...): resolve release-blocking defect DEF-xxx
docs(deploy): add environment deployment and rollback runbook
docs(rtm): link requirements design implementation and results
release: prepare 1.0.0-rc.1
```

Day 4 exit gate: Release candidate builds and deploys; all Must requirements have test results; no unresolved Critical defects; High defects are fixed or explicitly risk-accepted; rollback is documented; RTM has no unexplained Must-requirement gaps.

12.5 Day 5 - Tuesday, 18 August 2026

Theme: Final regression, controlled release, reporting, demonstration, and evaluation.

Team objectives: Freeze the product, verify the release, integrate academic evidence, rehearse presentation/demo, and archive a reproducible submission.

Member	Assigned work	Required evidence
M1	Chair release gate; integrate executive report sections; finalize contribution and lessons-learned records	Approval minutes, report, contribution matrix
M2	Finalize user manual, screenshots, requirements narrative, slides, and user-facing demo data	Reviewed guide and slide segment
M3	Verify all design diagrams/data dictionary against final release; finalize architecture slides and viva notes	Design consistency checklist
M4	Run final regression and smoke tests; finalize Test Report, Defect Summary, RTM, and quality metrics	Signed test summary and green result links
M5	Verify clean clone/build/deploy, tag release, generate release notes, archive checksums and final package	v1.0.0-se-class, release page, archive manifest

Release sequence:

1. Freeze features and accept documentation/test fixes only.
2. Pull a clean copy and install from the lock file.
3. Run type checking, lint, tests, and production build.
4. Run smoke tests against the release candidate.
5. Confirm secrets and personal data are absent.
6. Confirm documents, diagrams, RTM, test report, user guide, and slides are current.
7. Record professor-ready demo data and reset procedure.
8. Approve the release in signed meeting minutes.
9. Merge release PR and tag v1.0.0-se-class.
10. Rehearse the timed presentation, live demo, backup demo, and viva transitions.

Required outputs:

- Final Project Report and appendices.
- PRD, SRS, SDD, Test Report, RTM, User Manual, Deployment and Maintenance documents.
- Final diagrams in editable and exported formats.
- Presentation slides, speaker allocation, demo script, and viva question bank.
- Contribution report based on issues, commits, PRs, reviews, artifacts, and speaking roles.
- Final release, release notes, checksum/archive manifest, and submission checklist.

Suggested commits:

```
docs(report): integrate final SE project report
docs(user-guide): finalize guided UNI-TEL workflows
test(regression): record final release verification
docs(presentation): add final slides demo and viva guide
docs(contributions): summarize verified team work
release: publish v1.0.0-se-class
```

Day 5 exit gate: Every final checklist item is signed; CI is green; clean build and deployment are verified; RTM is complete; release is tagged; all members can explain both theory and their evidence.

Chapter 13

Required Software Engineering Artifact Set

Each artifact must have a title, version, authors, reviewers, approval, revision history, table of contents where appropriate, and links to related requirements or evidence.

No.	Artifact	Minimum content	Owner	Due
01	Project Proposal and Charter	Problem, vision, objectives, stakeholders, scope, constraints, success, approvals	M1	D1
02	Feasibility Study	Technical, economic, operational, schedule, legal/security conclusions	M1	D1
03	Methodology Study	Lifecycle theory comparison, selection criteria, ADR-001	M1	D1
04	Project Management Plan	WBS, schedule, effort, roles, RACI, communications, quality, procurement/tools	M1/M5	D1
05	PRD	Users, needs, value, features, exclusions, priorities, metrics, release criteria	M1	D2
06	SRS	Introduction, overall description, FRs, NFRs, interfaces, rules, constraints, acceptance	M2	D2
07	Use-Case Specification	Actors, diagram, pre/postconditions, main/alternate/exception flows	M2	D2
08	Software Design Document	Architecture, modules, data flows, interfaces, security, errors, decisions	M3	D3
09	Diagram Set	Context, use case, DFD 0/1, activity, sequence, component, ERD, deployment	M3	D3
10	Database Design	Tables, keys, constraints, indexes, RLS, dictionary, migration strategy	M3	D3
11	UI/UX Specification	Navigation, screen inventory, validation, states, accessibility, responsive rules	M2	D3
12	Test Plan	Scope, levels/types, environment, data, entry/exit, roles, reporting	M4	D2
13	Test Cases and Results	Preconditions, inputs, steps, expected/actual, status, evidence	M4	D4
14	Defect Log and Test Report	Severity, lifecycle, resolution, retest, metrics, residual risk	M4	D5
15	RTM	Requirement to use case, design, implementation, test, result, evidence	M4	D5
16	Risk Management Plan	Identification, scoring, response, owner, trigger, residual risk	M1/M4	Daily
17	SCM and Change Plan	Baselines, Git flow, reviews, versions, builds, changes, audits	M5	D2

No.	Artifact	Minimum content	Owner	Due
18	CI/CD and Deployment Guide	Pipeline, variables, build, deploy, smoke, rollback, recovery	M5	D4
19	Maintenance Plan	Corrective, adaptive, perfective, preventive work; support and monitoring	M5	D5
20	User Manual	Setup/login, workflows, validation, troubleshooting, screenshots	M2	D5
21	Final Project Report	Integrated SE narrative, evidence, results, limitations, learning, references	All	D5
22	Presentation and Demo	Theory-to-evidence slides, timed script, backup evidence, viva preparation	All	D5
23	Team Evidence Pack	Minutes, decisions, board, commits, PRs, reviews, daily logs, contributions	M1	D5

Chapter 14

Requirements Engineering Standard

14.1 ID scheme

BUS-xx	Business objective
USR-xx	User need
FR-xx	Functional requirement
NFR-xx	Non-functional requirement
BR-xx	Business rule
UC-xx	Use case
AC-xx	Acceptance criterion
TC-xx	Test case
DEF-xx	Defect
RISK-xx	Risk
CR-xx	Change request
ADR-xxx	Architecture/project decision record

14.2 Seed functional requirements

These are starting requirements. Member 2 must verify them against the baseline and scope before the Day 2 review.

ID	Requirement	Priority
FR-01	The system shall allow a student to register, authenticate, end a session, and recover from authentication errors safely.	Must
FR-02	The system shall permit authenticated users to access only their own academic records.	Must
FR-03	The student shall create, view, edit, and delete semesters subject to integrity rules.	Must
FR-04	The student shall manage subjects, credits, and grades within a selected semester.	Must
FR-05	The student shall record and update subject attendance values, and the system shall calculate the percentage.	Must
FR-06	The student shall record examinations, totals, obtained marks, and weightage subject to validation.	Must
FR-07	The system shall calculate and display valid marks percentages and weighted performance.	Must
FR-08	The system shall calculate and display semester SGPA and cumulative CGPA from approved grade rules.	Must
FR-09	The system shall display academic summaries, trends, distributions, and warnings supported by stored data.	Must
FR-10	The system shall import supported academic data and report invalid input without corrupting existing records.	Should

ID	Requirement	Priority
FR-11	The system shall export supported academic data or reports in the formats implemented by the release.	Should
FR-12	The system shall present notifications or warnings for relevant academic conditions implemented by the release.	Should
FR-13	The student shall view and update supported profile and preference information.	Could

14.3 Seed non-functional requirements

Values are release targets. Results must be measured under a documented test environment.

ID	Quality attribute	Measurable requirement
NFR-01	Security	Unauthenticated protected routes redirect to authentication, and database RLS prevents cross-user CRUD access in executed two-user tests.
NFR-02	Data integrity	Marks satisfy <code>0 <= obtained <= total</code> , total is positive, credits and percentages satisfy documented ranges, and invalid inputs are rejected.
NFR-03	Reliability	No uncaught application failure occurs during the ten critical end-to-end scenarios; recoverable errors show a usable message.
NFR-04	Performance	On the documented test device/network, primary post-login pages become usable within 3 seconds for the defined sample dataset.
NFR-05	Usability	A first-time student can complete semester, subject, attendance, and marks workflows using the user guide without facilitator correction in the recorded UAT.
NFR-06	Accessibility	Critical forms have programmatic labels, keyboard-reachable controls, visible focus, and no serious automated accessibility finding in the selected audit tool.
NFR-07	Compatibility	Critical journeys pass on current Chrome and Edge desktop and one mobile viewport at 360 x 800 or a documented equivalent.
NFR-08	Maintainability	Type check, lint, automated tests, and production build run from documented commands and pass in CI.
NFR-09	Privacy	Test evidence and the repository contain no real student records, credentials, tokens, or secret environment values.
NFR-10	Recoverability	Deployment and data-change procedures state rollback/recovery actions, ownership, and verification steps.

14.4 Requirement quality checklist

Each approved requirement must be:

- Correct and necessary.
- Atomic rather than combining unrelated behavior.
- Unambiguous and written with **shall** for mandatory behavior.
- Feasible in the five-day scope.
- Verifiable through inspection, analysis, demonstration, or test.
- Prioritized and uniquely identified.
- Traceable backward to a need and forward to design, implementation, and tests.
- Consistent with every other approved requirement.

14.5 Critical use cases

At minimum, fully specify:

- UC-01 Register and authenticate.
- UC-02 Manage a semester.
- UC-03 Manage subjects and credits.
- UC-04 Record attendance.
- UC-05 Record examination marks.
- UC-06 View SGPA/CGPA and analytics.
- UC-07 Import academic data.
- UC-08 Export an academic report.

Each description includes actor, trigger, preconditions, postconditions, main flow, alternate flow, exception flow, rules, requirement IDs, and acceptance tests.

Chapter 15

Analysis and Design Standard

15.1 Architecture boundary

Student browser

- > React pages and reusable UI components
- > hooks, validation, domain utilities, and services
- > Supabase client/API
- > Supabase Authentication and PostgreSQL with RLS
- > hosted frontend and managed backend deployment

15.2 Required diagrams and content

Diagram	Must show	Theory demonstrated
System context	Student, UNI-TEL boundary, Supabase/auth/data, export target, information flows	System boundary and external entities
Use-case	Student actor and approved user goals; relationships only where semantically valid	Functional analysis
DFD Level 0	Major processes, external entity, data stores, named data flows	Logical process modeling
DFD Level 1	Decomposition of academic record management or analytics	Functional decomposition and balancing
Activity	Decisions, validation, success/failure for marks or attendance entry	Workflow behavior
Sequence - login	Browser/UI, auth hook/service, Supabase Auth, protected route	Time-ordered interaction
Sequence - academic record	UI, hook/service, Supabase, database, response/error	Interface responsibility
Component	Pages, academic components, hooks/services, integrations, data platform	Modular architecture
ERD	Profile, semester, subject, attendance, marks, keys/cardinality	Data modeling
Deployment	User device, Vercel/static hosting, Supabase Auth/API/PostgreSQL	Physical architecture

15.3 Design consistency rules

- Entity and process names must match the SRS, data dictionary, code, and RTM.

- DFDs show data flow, not control flow or UI navigation.
- Use-case diagrams show user goals, not every button.
- Sequence messages correspond to real interfaces or clearly labelled conceptual operations.
- ERD cardinality and optionality match verified foreign keys and constraints.
- Deployment diagrams distinguish logical components from physical/runtime nodes.
- Every significant decision has an ADR with context, alternatives, decision, and consequences.

15.4 Database review focus

- Treat migration files as authoritative until verified against a deployed schema.
- Reconcile or remove misleading standalone schema snapshots through a reviewed task.
- Verify primary/foreign keys, unique constraints, check constraints, generated values, indexes, cascades, triggers, and RLS policies.
- Confirm calculations avoid division by zero and invalid numeric ranges.
- Test with two synthetic users to demonstrate data isolation.
- Document migration ordering, backup, rollback, and destructive-change risks.

Chapter 16

Implementation and Test Strategy

16.1 Product-hardening principle

The five-day project is not a feature expansion. Technical changes must close a verified gap in an approved requirement, quality target, security control, test, schema definition, accessibility review, documentation-to-code consistency check, or release process.

16.2 Planned automated test areas

Area	Representative checks	Suggested location
Grade calculations	Grade point mapping, weighted credits, empty semester, invalid grade	<code>tests/unit/gradeCalculations.test.ts</code>
Marks calculations	Valid percentage, weightage boundary, zero total rejection, obtained greater than total	<code>tests/unit/marksCalculations.test.ts</code>
Attendance	Percentage, zero classes, attended greater than held, threshold behavior	<code>tests/unit/attendanceCalculations.test.ts</code>
Semester normalization	Normal values, imported strings, invalid values, regression for 7 becoming 70	<code>tests/unit/semesterNormalization.test.ts</code>
Validation service	Required text, numeric ranges, exam type, cross-field validation	<code>tests/unit/validationService.test.ts</code>
Protected routing/auth	Unauthenticated redirect, authenticated access, loading state	<code>tests/integration/protectedRoute.test.tsx</code>
Import	Valid fixture, malformed JSON, invalid records, transactional/partial behavior	<code>tests/integration/importAcademicData.test.ts</code>

16.3 Test levels and types

- **Static verification:** reviews, TypeScript checks, ESLint, document consistency.
- **Unit testing:** calculations, normalization, and validation in isolation.
- **Integration testing:** hooks/services with Supabase boundary or controlled mocks; database constraints/RLS in a safe environment.
- **System testing:** complete browser-level user journeys.
- **Regression testing:** previously fixed semester import and all release-critical workflows.
- **Security testing:** authentication, authorization/RLS, input handling, secrets, dependency review.
- **Usability/accessibility testing:** task completion, navigation, labels, focus, contrast/audit findings.
- **Compatibility testing:** supported desktop browsers and selected mobile viewport.
- **Performance testing:** documented page/task measurements on defined data and environment.
- **User acceptance testing:** student-perspective scenarios with recorded results.

16.4 Test-case record

Every executed case records:

Field	Example
ID	TC-FR06-01
Requirement	FR-06, NFR-02
Objective	Verify valid marks are saved and percentage is calculated
Preconditions	Authenticated synthetic user; Semester 1 exists
Data	Subject DBMS, obtained 42, total 50, weightage 20
Steps	Open Marks; add record; enter data; save; reopen record
Expected	Record persists; raw percentage is 84%; weighted contribution follows the documented formula
Actual	Enter the observed output after execution
Status	Pass, Fail, Blocked, or Not Run
Evidence	Screenshot/log/CI link and execution date
Executor/reviewer	Named member and reviewer

16.5 Defect severity

Severity	Definition	Release response
Critical	Data exposure/loss, authentication bypass, build/deploy impossible, core app unavailable	Must fix before release
High	Must-have workflow fails or calculation is materially wrong with no safe workaround	Fix or formally stop release
Medium	Important behavior degraded with a usable workaround	Fix if time permits; document residual risk
Low	Cosmetic, wording, or minor usability issue	Record for maintenance backlog

16.6 Quality commands

The DevOps and QA leads must make these concepts executable from `package.json` and CI:

```
npm ci
npm run type-check
npm run lint
npm run test -- --run
npm run build
```

Record the exact tool versions, runner OS, command output, date, and commit SHA in the Test Report.

Chapter 17

Traceability Strategy

The RTM is the central proof that theory is displayed through the application.

Requirement	Need/use case	Design	Implementation	Test	Result/evidence
FR-01	UC-01	Login sequence, auth architecture	AuthPage, useAuth, ProtectedRoute	TC-FR01-01/02	Executed result link
FR-03	UC-02	Semester activity/ERD	Semester page/hook/service, semesters table	TC-FR03-01/02	Executed result link
FR-05	UC-04	Attendance activity/ERD	Attendance editor/service, attendance_records	TC-FR05-01/02	Executed result link
FR-06	UC-05	Marks sequence/ERD	Marks editor/service, marks_records	TC-FR06-01/02	Executed result link
FR-08	UC-06	Calculation design	Grade utilities, semester aggregation	TC-FR08-01/02	Executed result link
NFR-01	UC-01 and all data use cases	Security/RLS design	Protected routes and RLS policies	TC-SEC-01/02	Executed result link

Before release, the team must calculate:

- Requirements coverage = requirements with executed tests / approved requirements x 100.
- Must-have pass rate = passed Must-have tests / executed Must-have tests x 100.
- Defect closure = closed defects / recorded defects x 100.
- Traceability completeness = requirements with need, design, implementation, test, and result links / approved requirements x 100.

Targets: 100% traceability and executed-test coverage for Must requirements; 100% closure of Critical and High release-blocking defects.

Chapter 18

Project Estimation and Resource Plan

Gross capacity is 5 members x 5 days x 8 hours = 200 person-hours. Reserve time for coordination and unexpected work.

Activity	Person-hours	Percentage
Initiation, governance, feasibility, planning	20	10%
Requirements and product documentation	32	16%
Analysis, architecture, database, diagrams	36	18%
Targeted implementation and automation	40	20%
Testing, defects, traceability, quality	36	18%
Deployment, release, maintenance docs	16	8%
Final report, slides, demo, viva	20	10%
Total	200	100%

This is a planning allocation. Record actual effort daily without fabricating precision. If capacity is lost, reduce Should/Could items before compromising Must-have testing or traceability.

Chapter 19

Risk Management

Score Probability (P) and Impact (I) from 1 to 5. Exposure = P x I.

ID	Risk	P	I	Score	Response and trigger	Owner
RISK-01	Five-day schedule causes incomplete artifacts	4	5	20	Freeze scope; daily gates; trigger if any Must item slips by one day	M1
RISK-02	Artificial-looking repository history harms credibility	3	5	15	Transparent baseline; genuine issues/PRs; trigger on request to backdate/split old work	M1/M5
RISK-03	Existing code and documentation disagree	4	4	16	Baseline audit; traceability review; trigger on unsupported README claim	M2/M3
RISK-04	Schema snapshots are duplicated or inconsistent	4	5	20	Use migrations as authority; verify deployed schema; trigger on ERD mismatch	M3
RISK-05	Missing automated tests allow calculation regression	4	5	20	Prioritize calculation/validation tests; trigger on changed domain utility	M4
RISK-06	Supabase or hosting access fails	3	4	12	Local build, screenshots, recorded backup demo, documented recovery	M5
RISK-07	Secrets or real student data enter Git/evidence	2	5	10	Synthetic data, secret scan, <code>.gitignore</code> , repository audit before release	M5/M4
RISK-08	Merge conflicts and unstable main branch	3	4	12	Small PRs, WIP limit, integration checkpoints, branch protection	M5
RISK-09	Unequal contribution or review bottleneck	3	4	12	RACI, review rotation, contribution dashboard, daily rebalance	M1
RISK-10	Live demo network or account failure	3	5	15	Seeded demo account, local/recorded backup, screenshots, rehearsal	M2/M5
RISK-11	Documentation consumes all technical time	3	4	12	Reuse verified evidence, parallel roles, YAGNI, fixed artifact outlines	M1
RISK-12	LaTeX/report compilation fails near deadline	2	3	6	Compile daily, keep PDF snapshot, freeze formatting early	M5

Review risks at the daily retrospective. Record status, mitigation work, changes in score, and residual risk at release.

Chapter 20

Change Management

After the Day 2 requirements baseline, every scope or requirement change uses a Change Request containing:

- Change ID and requester.
- Date and reason.
- Affected requirement, design, code, test, document, schedule, and risk.
- Alternatives considered.
- Effort and dependency impact.
- Decision: Approved, Rejected, or Deferred.
- Approver and decision date.
- Links to issues, PRs, tests, and RTM updates.

Approval rule:

- Defect correction that restores an approved requirement: Product Lead and relevant technical lead.
- New feature or changed requirement: Product Lead plus affected leads; it must not endanger Must items.
- Database/security/release change: mandatory QA and DevOps review.
- Could-have change after Day 3: defer to maintenance backlog.

Chapter 21

Quality Assurance Plan

21.1 Review checklists

Requirements review: complete, correct, feasible, singular, unambiguous, testable, prioritized, traceable.

Design review: requirements coverage, clear boundaries, consistent names, valid data relationships, security/error paths, deployability, testability.

Code/PR review: linked issue, focused diff, correctness, validation, security, tests, types, error handling, no secrets, documentation and RTM updated.

Test review: requirement link, deterministic setup, positive/negative/boundary coverage, actual result, evidence, reproducibility, defect link for failures.

Release review: green CI, clean build, smoke/regression pass, no blocking defect, complete RTM, current docs, recoverability, final archive.

21.2 Project metrics

Report values with context, not vanity numbers:

- Approved requirements by priority.
- Requirements with design/test/result traceability.
- Test cases planned/executed/passed/failed/blocked.
- Automated test count and meaningful coverage areas.
- Defects by severity, state, and mean resolution time where useful.
- Pull requests opened/approved/merged and substantive reviews.
- Build success and release-candidate status.
- Planned versus completed backlog items.
- Contribution evidence by category, not commits alone.

Chapter 22

Deployment, Operations, and Maintenance

22.1 Environment model

Environment	Purpose	Data	Change control
Local development	Individual coding and unit testing	Synthetic/local test data	Feature branches
Test/staging	Integration, RLS, UAT, release candidate	Synthetic shared test data	Reviewed <code>main</code> or release branch
Production/demo	Professor demonstration and final evidence	Seeded non-personal demo data	Approved release only

22.2 Deployment guide must include

- Prerequisites and supported Node/npm versions.
- Clean clone and `npm ci` procedure.
- Environment-variable names with safe example values.
- Supabase migration ordering and verification.
- Local build, preview, and smoke-test commands.
- Hosting configuration and deployment steps.
- Release approval, version/tag, and release notes.
- Rollback procedure for frontend and database changes.
- Backup/recovery and data-retention assumptions.
- Known limitations, service dependencies, and troubleshooting.

22.3 Maintenance categories

- **Corrective:** fix defects found after release.
- **Adaptive:** respond to browser, dependency, Supabase, or hosting changes.
- **Perfective:** improve usability, performance, or supported reporting.
- **Preventive:** dependency updates, refactoring, additional tests, documentation refresh.

The maintenance backlog should contain deferred Medium/Low defects and out-of-scope enhancements without presenting them as completed release functionality.

Chapter 23

Final Report Structure

1. Cover page, certificate/declaration if required, acknowledgements.
2. Abstract and executive summary.
3. Problem background, objectives, stakeholders, scope, limitations.
4. Feasibility study.
5. SDLC and methodology theory comparison.
6. Selected RAD/iterative/Kanban/V-Model/DevOps approach and five-day evidence.
7. Project planning: WBS, schedule, estimates, RACI, communication, risks.
8. Requirements engineering: PRD, SRS, actors, use cases, FRs/NFRs, priorities.
9. Analysis and design: architecture, DFD/UML, database, UI, security, decisions.
10. Implementation overview: modules, technologies, targeted five-day improvements.
11. Verification and validation: strategy, cases, automation, results, defects, RTM.
12. Configuration management, Git collaboration, CI/CD, release and deployment.
13. Maintenance, limitations, ethical/privacy considerations, future scope.
14. Results against success criteria and lessons learned.
15. Conclusion.
16. References using one consistent citation style.
17. Appendices: plans, minutes, decisions, test evidence, contribution matrix, user guide.

Every theory chapter must end with a short **Application to UNI-TEL** subsection containing real repository evidence.

Chapter 24

Presentation and Demonstration Plan

24.1 Recommended 18-slide deck

Slides	Subject	Speaker
1-3	Title, problem, objectives, stakeholders, scope	M1
4-5	Methodology comparison, selection, five-day SDLC	M1
6-8	PRD/SRS, FR/NFR examples, use cases, priorities	M2
9-11	Architecture, DFD/UML, ERD/database/security	M3
12-14	Test strategy, defects, automation, RTM and metrics	M4
15-16	Git collaboration, CI/CD, deployment, maintenance	M5
17	Live demonstration with speaker hand-offs	All
18	Results, limitations, future scope, conclusion	M1 plus all for questions

24.2 Theory-to-evidence rule

Use the pattern:

Theory concept -> UNI-TEL decision/artifact -> implementation evidence -> test/result

Example:

Requirements traceability -> FR-06 -> MarksEditor + marks_records -> TC-FR06-01 Pass

24.3 Timed demo script

1. Show repository board, requirement ID, PR, CI, and RTM row (45 seconds).
2. Open the deployed landing page and authenticate (30 seconds).
3. Create or open a semester and show subjects/credits (45 seconds).
4. Enter attendance, including one validation example (45 seconds).
5. Enter marks and show calculated percentage/weightage (45 seconds).
6. Show SGPA/CGPA and analytics (45 seconds).
7. Export a supported report (30 seconds).
8. Explain RLS/user isolation and show corresponding test evidence (45 seconds).
9. Show final CI/release result and state limitations (30 seconds).

Prepare a seeded synthetic account, stable browser tabs, backup screenshots, and a short recorded demo in case network services fail.

24.4 Viva coverage by every member

Every member must be able to explain:

- Why RAD was selected and why Waterfall, Spiral, or pure Scrum was not.
- Difference between verification and validation.
- Difference between functional and non-functional requirements.
- How one UNI-TEL requirement is traced to design, code, test, and result.
- How authentication differs from authorization and how RLS helps.
- Difference between unit, integration, system, regression, and acceptance tests.
- How Git branches, PR review, CI, release, rollback, and maintenance work.
- A limitation, a risk, a defect, a design trade-off, and a lesson learned.

Chapter 25

Evidence and Contribution Management

25.1 Daily evidence folders

Each day folder should contain:

- Stand-up and review minutes.
- Board snapshot or exported issue list.
- Document and diagram review evidence.
- Relevant build/test output.
- PR and review links.
- Screenshots using synthetic data.
- Updated risk, decision, defect, and contribution records.
- Daily progress summary: planned, completed, carried, blocked, decision.

25.2 Contribution matrix

Record quality and type, not just quantity:

Owned		Technical changes	Authored artifacts	Tests/evidence	PR	Presentation
Member	issues				reviews	
M1	WP1/WP7 items	Auth/security verification	Charter, PRD, feasibility	Acceptance evidence	Reviews M5	Slides 1-5, 18
M2	WP2/UI items	Validation/accessibility	SRS, use cases, UI guide	UX evidence	Reviews M1	Slides 6-8
M3	WP3/data items	Schema/integrity	SDD, diagrams, dictionary	DB evidence	Reviews M2	Slides 9-11
M4	WP5 items	Test framework/defect fixes	Test docs, RTM	Test and security evidence	Reviews M3	Slides 12-14
M5	WP0/WP6 items	CI/deployment	SCM, deploy, maintenance	Build/release evidence	Reviews M4	Slides 15-16

Update with actual issue, PR, and commit links. Never assign authorship to someone who did not perform the work.

Chapter 26

Final Submission Structure

```
submission/  
|-- 01_Project_Charter_and_Proposal.pdf  
|-- 02_Feasibility_and_Methodology.pdf  
|-- 03_Project_Management_Plan.pdf  
|-- 04_PRD.pdf  
|-- 05_SRS_and_Use_Cases.pdf  
|-- 06_Software_Design_and_Diagrams.pdf  
|-- 07_Database_Design.pdf  
|-- 08_Test_Plan_Cases_and_Report.pdf  
|-- 09_Requirements_Traceability_Matrix.xlsx-or-pdf  
|-- 10_Risk_Change_and_Configuration_Management.pdf  
|-- 11_Deployment_Maintenance_and_User_Guide.pdf  
|-- 12_Final_Project_Report.pdf  
|-- 13_Presentation.pdf  
|-- 14_Team_Contribution_and_Evidence.pdf  
|-- source-code-and-document-sources/  
|-- release-notes/  
`-- README_Submission_Index.pdf
```

Do not duplicate contradictory content across documents. The Submission Index identifies the authoritative version and location of every artifact.

Chapter 27

Final Audit Checklists

27.1 SE completeness

- ☐ Problem, vision, objectives, stakeholders, scope, exclusions, assumptions, constraints.
- ☐ Technical, operational, economic, schedule, legal/security feasibility.
- ☐ Methodology theory comparison and justified selection.
- ☐ WBS, schedule/Gantt, estimates, roles, RACI, communications, governance.
- ☐ PRD, SRS, actors, use cases, FRs, NFRs, rules, priorities, acceptance criteria.
- ☐ Context, use-case, DFD 0/1, activity, sequence, component, ER, deployment diagrams.
- ☐ Architecture, modules, interfaces, database, UI, validation, errors, security design.
- ☐ Implementation evidence tied to approved issues and requirements.
- ☐ Static, unit, integration, system, regression, security, UX, compatibility, UAT evidence.
- ☐ Test Plan, cases/results, Defect Log, Test Report, and complete RTM.
- ☐ Risk, change, configuration, version, review, and quality management.
- ☐ CI/CD, environment, deployment, rollback, recovery, maintenance, and user guide.
- ☐ Final report, presentation, demo, viva, contribution record, and release archive.

27.2 Repository and release

- ☐ Imported baseline is transparent and old `.git` history is absent.
- ☐ No credentials, `.env`, personal data, build output, or temporary evidence is tracked.
- ☐ Issues, branches, commits, PRs, and reviews are genuine and linked.
- ☐ `npm ci`, type check, lint, automated tests, and build pass on the final SHA.
- ☐ Database migration and RLS evidence are verified safely.
- ☐ All Critical/High release blockers are resolved or release is stopped.
- ☐ RTM covers all Must requirements with executed results.
- ☐ Clean deployment and smoke test are successful.
- ☐ Tag `v1.0.0-se-class` and release notes exist.
- ☐ Submission archive can be opened and understood on another machine.

27.3 Presentation readiness

- ☐ Every theory claim has project evidence.
- ☐ All five members have balanced speaking roles and can answer common viva questions.
- ☐ Diagrams and screenshots are legible from presentation distance.
- ☐ Demo account uses synthetic data and has been reset.
- ☐ Live demo is rehearsed and timed.
- ☐ Offline screenshots/video and local build are available as backup.
- ☐ Limitations and future scope are stated honestly.

Chapter 28

Reusable Record Templates

28.1 Daily progress record

Date: 14 August 2026

Iteration goal: Establish verified baseline, scope, governance, and approved Day 2 work.

Completed: List issue/PR/document links.

Carried forward: State item, reason, owner, and new due date.

Blocked: State blocker, impact, owner, and escalation.

Decisions: Link ADR or meeting decision.

Risks changed: Link Risk Register entries.

Quality result: Record build/test/document gate.

Approvals: Project Lead and rotating reviewer.

28.2 Meeting minutes record

Meeting: Day 2 Requirements Review

Date/time: 15 August 2026, 17:30 IST

Attendees: Members 1-5

Inputs: PRD v1.0, SRS review candidate, Use Cases, Test Plan v0.1

Decisions: Record approved/rejected requirement and reason.

Actions: Record owner, due date, and issue link.

Risks/changes: Record Risk or Change Request ID.

Approval: Member 1 with reviewer confirmation.

28.3 Architecture decision record

ADR-001: Use RAD as the primary five-day lifecycle

Status: Accepted

Context: Five calendar days, existing prototype, five-person team, full SE evidence required.

Options: Waterfall, Scrum, Spiral, RAD with iterative/Kanban/DevOps practices.

Decision: RAD with daily increments, Kanban flow, V-Model traceability, and CI.

Consequences: Fast execution and reuse; strict gates are required to prevent documentation and quality g

28.4 Change request example

CR-01: Add teacher dashboard

Reason: Suggested extension during Day 3.

Impact: New actor, roles, RLS policies, screens, requirements, tests, and schedule risk.

Decision: Deferred.

Justification: Outside the frozen student-only scope and endangers Must-have verification.

Target: Maintenance/future-scope backlog.

Approver: Project Lead after architecture and QA consultation.

28.5 Defect example

DEF-01: Invalid obtained marks can exceed total marks

Requirement: FR-06, NFR-02

Severity: High

Environment/commit: Record exact test environment and SHA.

Reproduction: Create marks record with obtained 60 and total 50.

Expected: Submission is rejected with a clear validation message.

Actual: Record observed behavior after execution.

Owner/reviewer: Assigned developer and QA reviewer.

Resolution/retest: Link fix PR and executed regression result.

Chapter 29

Immediate Start Checklist

Complete these actions in order today:

1. Create the new private/public repository according to class rules.
2. Import the current project as the single transparent baseline commit.
3. Verify `.gitignore`, `.env.example`, licenses, and absence of secrets.
4. Confirm a clean `npm ci` and `npm run build` from the new repository.
5. Add all five collaborators and protect `main`.
6. Create the milestone, Kanban columns, labels, issue and PR templates.
7. Replace Member 1-5 labels with names and confirm review rotation.
8. Create Day 1 issues SE-001 through SE-006 with acceptance criteria.
9. Run the baseline demonstration and parallel audits.
10. Finish the Day 1 gate before starting requirements work.

Chapter 30

Approval

The team approves this plan when all five members agree to the scope, methodology, roles, contribution rules, five-day schedule, evidence policy, and release gates. Any later change must follow the Change Management section.

Role	Approval responsibility
Member 1 / Project Lead	Scope, schedule, release decision
Member 2 / BA-UX Lead	Requirements and usability completeness
Member 3 / Architecture-Data Lead	Design and data correctness
Member 4 / QA-Security Lead	Verification, defects, and traceability
Member 5 / DevOps-Release Lead	Configuration, build, deployment, archive

Planning principle: A credible five-day project is demonstrated by a transparent baseline, disciplined decisions, traceable artifacts, tested corrections, meaningful peer review, and a reproducible release. The goal is not to imitate months of coding history; it is to show that the team can apply the complete Software Engineering process to a real system under a strict time box.